TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

BASE DE DATOS II

UNIDAD 5 · CLASE 1 de 2

El administrador de la base: usuarios, roles, permisos y auditoría

Administración y Seguridad aplicadas al proyecto integrador Food Store

Asignatura	Base de Datos II, Unidad 5: Administración y Seguridad
Ubicación	Semana 9 de la cursada, Clase 1 de 2
Motor	PostgreSQL 16 o superior
Proyecto	Food Store: capa de administración y acceso
Meta de la clase	Diseñar el esquema de roles y permisos de mínimo privilegio de Food Store, y activar una política de auditoría básica
Ciclo lectivo	2026

Apunte teórico de cátedra · Material para el aula virtual. La consigna del trabajo práctico integrador de la unidad se distribuye en documento aparte, al cierre de la Clase 2.

Presentación de la unidad y de esta clase

En las cuatro unidades anteriores, todo el trabajo sobre Food Store se hizo desde una única identidad con privilegios plenos: la sesión con la que se creó el esquema, se cargaron los datos, se escribieron las consultas y se midieron los índices. Esa comodidad no existe en producción. Una base real es accedida por muchas identidades distintas al mismo tiempo (la aplicación web que atiende a los clientes, el panel de administración, el proceso que arma los reportes gerenciales, el equipo de soporte, el propio equipo de desarrollo), y ninguna de ellas debería poder hacer más de lo que su tarea exige. Administrar una base de datos es, en una parte sustancial, administrar quién puede hacer qué.

La Unidad 5 se dicta en dos clases. Esta primera clase construye el modelo de identidades y permisos: quién puede conectarse, qué puede hacer una vez conectado, y cómo queda un registro confiable de lo que hizo. La segunda clase se ocupa de qué pasa cuando ese modelo falla o es atacado (inyección de SQL, escalamiento de privilegios y fuga de datos) y de un problema que atraviesa toda la operación de una base: los datos personales que contiene, cómo protegerlos, y qué implica eso, en particular, para el trabajo con inteligencia artificial.

Sobre el trabajo con inteligencia artificial en esta unidad

En seguridad, la asimetría entre el daño y el beneficio de automatizar sin verificar es mayor que en cualquier otra unidad de la materia: un índice innecesario se detecta porque encarece las escrituras, mientras que un permiso innecesario pasa inadvertido hasta que alguien lo utiliza. Por eso el criterio de la cátedra es el mismo que en unidades anteriores, aplicado con mayor rigor: la IA propone el esquema de roles y permisos; el administrador lo audita línea por línea contra el principio de mínimo privilegio antes de aplicarlo, y la decisión final queda documentada.

1. El rol del administrador de bases de datos

1.1. De «quien escribe consultas» a «quién responde por los datos»

Hasta este punto, el trabajo de la cursada se pareció al de un desarrollador que consulta y modela datos. El administrador de bases de datos (*DBA, por Database Administrator*) tiene una responsabilidad más amplia: no solo que las consultas funcionen y respondan rápido, sino que los datos estén disponibles cuando se los necesita, íntegros frente a errores y fallas, y accesibles únicamente para quien corresponde. Estos tres objetivos (disponibilidad, integridad y seguridad), sumados al rendimiento trabajado en las Unidades 2 y 3, resumen las responsabilidades del rol (Elmasri y Navathe, 2016).

En un equipo chico, la misma persona programa la aplicación y administra la base; en un equipo grande, son roles separados. En ambos casos, las decisiones de administración (a quién se otorga acceso, qué se audita y cómo se actúa ante un incidente) son de naturaleza distinta a las de desarrollo. Por ese motivo, deben tomarse de forma explícita y no como un agregado de último momento al desarrollo de la aplicación.

1.2. Las tareas del DBA en el mapa de la materia

El programa de la materia recorre las tareas centrales de un administrador de bases de datos. Ubicar la Unidad 5 en ese mapa permite comprender por qué la seguridad se estudia después de la optimización y antes del mantenimiento:

Tarea del DBA	Dónde se trabaja en la materia
Definir estructura e integridad	Unidades 1 y 4
Garantizar consistencia transaccional	Unidad 1
Optimizar el acceso a los datos	Unidades 2 y 3

Tarea del DBA	Dónde se trabaja en la materia
Administrar identidades y permisos	Unidad 5, Clase 1
Prevenir y responder a amenazas; proteger datos personales	Unidad 5, Clase 2
Mantener la base operativa: respaldo, monitoreo	Unidad 6

1.3. El caso de Food Store

El caso de Food Store
Todo el esquema del proyecto se construyó, hasta hoy, con un único rol dueño de los objetos: el que ejecuta CREATE TABLE, CREATE INDEX, CREATE VIEW. En producción, ese rol jamás debería ser el que usa la aplicación web día a día. Si las credenciales de la aplicación se filtran, que es uno de los escenarios más frecuentes, un atacante que se conecta con el rol dueño del esquema puede borrar tablas, alterar precios o leer cualquier dato. Esta clase construye los roles reales que Food Store necesita para que eso no sea posible.

2. Usuarios y roles en PostgreSQL

2.1. Todo es un ROLE

Desde la versión 8.1, PostgreSQL unificó dos conceptos que en otros motores siguen separados: usuario y grupo. Internamente existe un único tipo de objeto, el ROLE, que puede tener permiso para iniciar sesión o no, y que puede contener a otros roles como miembros. La sentencia CREATE USER es un alias de CREATE ROLE ... LOGIN: produce el mismo resultado, con un nombre más familiar (PostgreSQL Global Development Group, 2005, 2023b, 2023d).

Esta unificación simplifica el modelo conceptual: no existen dos sistemas paralelos, uno de usuarios y otro de grupos, con reglas propias. Hay roles, y algunos de esos roles se usan para agrupar permisos (roles «**de grupo**», sin inicio de sesión) mientras que otros se usan para conectarse (roles «**de login**», con contraseña).

2.2. Crear roles: sintaxis y atributos

```
-- Un rol de grupo: agrupa permisos, nadie se conecta "como" él
CREATE ROLE rol_app_lectura;
```

```
-- Un rol de login: representa una conexión real
CREATE ROLE app_web WITH LOGIN PASSWORD 'una_contrasena_fuerte';
ALTER ROLE app_web VALID UNTIL '2027-01-01';
ALTER ROLE app_web CONNECTION LIMIT 50;
```

- **LOGIN / NOLOGIN**: si el rol puede o no abrir una conexión por sí mismo.
- **PASSWORD**: el motor almacena la contraseña como hash (SCRAM-SHA-256 por omisión); nunca se ve en texto plano después de creada.
- **SUPERUSER / NOSUPERUSER**: un superusuario omite todas las verificaciones de permisos. Ningún rol de aplicación debe tener este atributo.
- **CREATEDB, CREATEROLE**: habilitan crear bases o crear otros roles; son atributos administrativos, no de aplicación.
- **VALID UNTIL, CONNECTION LIMIT**: límites operativos: vencimiento de la credencial y tope de conexiones simultáneas, útil para contener un uso anómalo (PostgreSQL Global Development Group, 2023b).

Error frecuente de alto impacto
Crear el rol de la aplicación con SUPERUSER "para que no dé problemas de permisos" anula todas las medidas que se describen en esta clase. Se trata de uno de los errores más frecuentes y de mayor impacto entre quienes comienzan a administrar una base de datos.

2.3. Roles de login vs. roles de grupo

La buena práctica es separar la definición de los permisos (roles de grupo, sin LOGIN, uno por función: lectura del catálogo, escritura operativa, soporte, reportes) de las credenciales que efectivamente se usan para conectarse (roles de login). El rol de login se hace miembro de uno o más roles de grupo con GRANT. La ventaja es que, cuando cambia de responsabilidad una persona o un servicio, se ajusta su membresía de grupo sin tocar la definición de los permisos, que se revisa una sola vez y queda estable (PostgreSQL Global Development Group, 2023d).

2.4. Diferencia entre el rol del motor y la columna de negocio

El esquema de Food Store ya tiene, desde la Unidad 1, una columna usuario.rol de tipo ENUM con los valores ADMIN y USUARIO. Esa columna no debe confundirse con los roles del motor que se estudian en esta clase.

	usuario.rol (columna)	ROLE de PostgreSQL
Capa	Aplicación / negocio	Motor de base de datos
Quién lo interpreta	El código de la aplicación	El propio PostgreSQL, en cada sentencia SQL
Qué decide	Qué pantallas o funciones mostrar al usuario final	Qué sentencias SQL puede ejecutar la conexión
Se puede leer con	SELECT rol FROM usuario	\du en psql, o information_schema.role_table_grants

Error frecuente
Pensar que basta con usuario.rol = 'ADMIN' para que ese cliente pueda saltarse las restricciones de la base es un error: si la conexión a la base la abre siempre el mismo rol técnico (app_web, por ejemplo), la columna rol no protege nada a nivel del motor, ya que es solo un dato que la aplicación interpreta. La autorización que el motor hace cumplir, incluso si la aplicación tiene un error, se define con GRANT/REVOKE o, para autorización fila por fila, con Row-Level Security (sección 3.6).

3. Permisos: el modelo GRANT

3.1. Objetos y privilegios

PostgreSQL define privilegios independientes para cada tipo de operación sobre cada tipo de objeto. Los más relevantes para el trabajo cotidiano de Food Store son los siguientes (PostgreSQL Global Development Group, 2023f, 2023h):

Privilegio	Sobre qué objetos	Qué habilita
SELECT	tabla, vista, columna	leer filas
INSERT	*tabla*	agregar filas nuevas
UPDATE	*tabla, columna*	modificar filas existentes
DELETE	*tabla*	borrar filas
TRUNCATE	*tabla*	vaciar la tabla entera de una vez, sin disparar triggers por fila; es más peligroso que DELETE

Privilegio	Sobre qué objetos	Qué habilita
REFERENCES	*tabla*	crear una clave foránea que apunte a ella
EXECUTE	*función, procedimiento*	ejecutarla
USAGE	*esquema, secuencia, tipo*	"poder ver y usar" el objeto contenedor; sin este privilegio, ningún objeto interno es alcanzable
CREATE	*esquema, base*	crear objetos nuevos dentro

3.2. GRANT y REVOKE

```
GRANT SELECT ON v_productos_vigentes, v_categorias_vigentes
    TO rol_app_lectura;

GRANT SELECT, INSERT, UPDATE ON pedido, detalle_pedido
    TO rol_app_escritura;

REVOKE DELETE, TRUNCATE ON pedido, detalle_pedido
    FROM rol_app_escritura;

-- Imprescindible y fácil de olvidar: sin USAGE sobre el esquema,
-- ningún otro privilegio sirve de nada.
GRANT USAGE ON SCHEMA public TO rol_app_lectura, rol_app_escritura;
```

Punto de partida: ningún privilegio
Un rol recién creado no tiene ningún privilegio sobre ningún objeto ajeno. Desde PostgreSQL 15, ni siquiera el esquema public otorga CREATE a los roles nuevos de forma automática (PostgreSQL Global Development Group, 2022). Cada acceso que un rol necesita se otorga explícitamente: es la contracara operativa del principio de mínimo privilegio que se desarrolla en la sección 4: ningún acceso se otorga por descuido, porque el punto de partida es cero (PostgreSQL Global Development Group, 2023f).

3.3. Permisos por columna

```
GRANT SELECT (id, nombre, apellido, mail, rol)
    ON usuario
    TO rol_soporte;
```

GRANT admite restringir el privilegio a un subconjunto de columnas. El resultado es equivalente al que ya se logró en la Unidad 3 con la vista v_usuarios_publico (que excluye contrasena y celular): el rol_soporte puede consultar la identidad de un usuario, pero nunca su contraseña. La cátedra prefiere resolver esto con vistas, porque son más legibles, se pueden probar de forma independiente y no

obligan a repetir la lista de columnas permitidas en cada GRANT. Sin embargo, el permiso por columna existe como alternativa directa que no requiere crear un objeto adicional (PostgreSQL Global Development Group, 2023f).

3.4. Herencia de roles

```
GRANT rol_app_lectura TO app_web;
GRANT rol_app_escritura TO app_web;

-- Un rol administrativo, miembro de todo, que opera acotado
-- a un único grupo cuando hace una tarea puntual:
SET ROLE rol_soporte;
-- ... acá adentro, la sesión solo puede lo que puede rol_soporte
...
RESET ROLE;
```

Cuando un rol es miembro de otro con el atributo INHERIT (*el comportamiento por omisión*), hereda automáticamente sus privilegios: no hace falta un paso adicional para activarlos. SET ROLE cumple la función opuesta: permite que una sesión con privilegios amplios se autolimite temporalmente a los de un rol de grupo, reduciendo la superficie de error mientras dura una tarea puntual. Es el mismo principio de mínimo privilegio aplicado a una sesión de trabajo (*PostgreSQL Global Development Group, 2023i*).

3.5. Permisos por defecto

```
ALTER DEFAULT PRIVILEGES FOR ROLE admin_datos IN SCHEMA public
    GRANT SELECT ON TABLES TO rol_reportes;
```

Un GRANT común solo aplica a los objetos que ya existen en el momento de ejecutarlo. Sin ALTER DEFAULT PRIVILEGES, cada tabla o vista nueva que se cree después requeriría otorgar el permiso a mano otra vez, y ese es el tipo de paso que suele omitirse bajo presión. Esta sentencia indica al motor que, a partir de ese momento, todo objeto nuevo que cree admin_datos en este esquema otorgue automáticamente SELECT a rol_reportes (*PostgreSQL Global Development Group, 2023a)*.

3.6. Row-Level Security (nociones)

Todo lo anterior filtra por columna y por tabla. La seguridad a nivel de fila (Row-Level Security, RLS) filtra por fila, según quién está consultando. Es la versión que el motor hace cumplir de algo que, hasta ahora, se resolvía escribiendo a mano WHERE usuario_id = ... en cada consulta de la aplicación, y que

un desarrollador puede omitir en alguna consulta, con consecuencias serias (*PostgreSQL Global Development Group, 2023k*).

```
ALTER TABLE pedido ENABLE ROW LEVEL SECURITY;

CREATE POLICY pedidos_propios ON pedido
    USING (usuario_id =
current_setting('app.usuario_actual')::bigint);
```

Con esta política activa, cualquier SELECT, UPDATE o DELETE sobre pedido que ejecute un rol sujeto a la política queda automáticamente restringido a las filas cuyo usuario_id coincide con el valor que la aplicación fijó al abrir la conexión (current_setting('app.usuario_actual')). Si la aplicación omite el filtro por usuario en una consulta, el motor lo aplica de todos modos. De este modo, la regla deja de depender de la disciplina de cada desarrollador y pasa a ser aplicada por el motor. RLS se retoma como ampliación optativa en el trabajo práctico de la unidad; estas nociones son suficientes para reconocer cuándo conviene proponerla.

4. El principio de mínimo privilegio

4.1. Qué es y por qué importa

El principio de mínimo privilegio establece que cada identidad (cada rol) debe tener exactamente el conjunto de permisos necesario para cumplir su función, ni uno más (Joint Task Force, 2020; Saltzer y Schroeder, 1975). Su finalidad es limitar el daño posible, tanto ante errores propios como ante ataques externos. Cuando se produce una falla, situación que en todo sistema en producción ocurre tarde o temprano, el alcance del daño queda limitado por lo que ese rol podía hacer. Un error de programación que intenta ejecutar un DELETE sin WHERE sobre pedido es rechazado por el motor si el rol no tiene privilegio de DELETE; el mismo error, ejecutado con un rol superusuario, borra todas las filas de la tabla.

4.2. Cómo se aplica en la práctica

- Roles por función, no por persona: no se crea un rol distinto por cada desarrollador con acceso amplio a todo; se crean roles que representan tareas (lectura del catálogo, escritura operativa, soporte, reportes) y las personas o servicios se agregan como miembros de esos roles.
- Revisión periódica: los permisos que se otorgaron para una tarea puntual y nunca se revocaron son la causa más común de erosión del mínimo privilegio con el tiempo (Joint Task Force, 2020).
- Nunca compartir una credencial de más privilegio del necesario para la tarea del momento: se prefiere SET ROLE (*sección 3.4*) a conectarse directamente como un rol amplio.

4.3. El caso de Food Store: diseño de roles

El caso de Food Store
Aplicando el criterio de las secciones anteriores al proyecto integrador, este es el esquema de roles que Food Store necesita para operar en condiciones reales:

Rol	Tipo	A qué accede	Para qué
rol_app_lectura	***grupo***	SELECT en v_productos_vigentes y v_categorias_vigentes	Servir el catálogo público del sitio
rol_app_escritura	***grupo***	SELECT/INSERT/UPDATE en pedido y detalle_pedido; EXECUTE en sp_crear_pedido; sin DELETE ni TRUNCATE	El flujo de compra de la aplicación
rol_soporte	***grupo***	SELECT en v_usuarios_publico y v_pedidos_resumen	Atención al cliente, sin ver contraseñas ni forzar cambios de stock
rol_reportes	***grupo***	SELECT en mv_ventas_categoria_mes y vistas analíticas	Panel gerencial, exclusivamente de lectura
app_web	***login***	miembro de rol_app_lectura + rol_app_escritura	La conexión real que usa el backend de la aplicación
admin_datos	***login***	dueño del esquema; sin SUPERUSER	El administrador de la base / la cátedra

4.4. El esquema de roles, consolidado

```
-- Roles de grupo: definen QUÉ se puede hacer
CREATE ROLE rol_app_lectura;
CREATE ROLE rol_app_escritura;
CREATE ROLE rol_soporte;
CREATE ROLE rol_reportes;

GRANT USAGE ON SCHEMA public
    TO rol_app_lectura, rol_app_escritura, rol_soporte,
rol_reportes;
```

```
GRANT SELECT ON v_productos_vigentes, v_categorias_vigentes
    TO rol_app_lectura;

GRANT SELECT, INSERT, UPDATE ON pedido, detalle_pedido
    TO rol_app_escritura;
GRANT EXECUTE ON PROCEDURE sp_crear_pedido(BIGINT, forma_pago,
JSONB)
    TO rol_app_escritura;

GRANT SELECT ON v_usuarios_publico, v_pedidos_resumen
    TO rol_soporte;

GRANT SELECT ON mv_ventas_categoria_mes
    TO rol_reportes;

-- Roles de login: definen QUIÉN se conecta
CREATE ROLE app_web WITH LOGIN PASSWORD 'cambiar_en_produccion';
GRANT rol_app_lectura, rol_app_escritura TO app_web;

CREATE ROLE admin_datos WITH LOGIN PASSWORD 'cambiar_en_produccion'
    CREATEDB CREATEROLE;
```

Este bloque es el que se espera encontrar en roles.sql al cierre de la unidad, ya sea copiado de este apunte y adaptado, o generado con asistencia de IA y verificado contra la tabla de la sección 4.3.

5. Auditoría de accesos y operaciones

5.1. Qué es auditar, y por qué no es lo mismo que loguear

Loguear es registrar eventos técnicos con fines de diagnóstico: qué consulta tardó mucho, qué conexión falló y por qué. Auditar es un objetivo distinto, aunque se apoye en mecanismos parecidos: dejar un registro con vocación de rendición de cuentas (quién hizo qué, cuándo y desde dónde) que además sea confiable, en el sentido de que quien realizó la acción no pueda alterar el registro que la documenta. Un log que vive en el mismo servidor y con el mismo usuario que podría querer borrar sus huellas no cumple ese segundo requisito; se retoma en la Unidad 6 al hablar de dónde se conservan los registros (Kent y Souppaya, 2006).

5.2. Mecanismos nativos

```
-- Qué tipo de sentencias quedan en el log del servidor
SHOW log_statement;
ALTER SYSTEM SET log_statement = 'mod';   -- ddl | mod | all | none

-- Conexiones y desconexiones
ALTER SYSTEM SET log_connections = on;
ALTER SYSTEM SET log_disconnections = on;

-- Quién está conectado ahora mismo, y qué está ejecutando
SELECT pid, usename, application_name, state, query
FROM   pg_stat_activity
WHERE  datname = current_database();
```

- **`log_statement`** controla qué sentencias se registran: ninguna, solo DDL, DDL más modificaciones (mod), o todas las sentencias (all, la opción que genera mayor volumen) (PostgreSQL Global Development Group, 2023e).
- **`pg_stat_activity`** es una vista del catálogo que muestra el estado de cada conexión activa en tiempo real; es el primer recurso a consultar ante una sospecha de actividad anómala en curso (PostgreSQL Global Development Group, 2023c).
- **`pg_stat_statements`** (extensión) acumula estadísticas agregadas por tipo de consulta: cuántas veces se ejecutó cada una y cuánto tardó en total, útil tanto para la optimización (Unidades 2 y 3) como para detectar un patrón de acceso fuera de lo normal (*PostgreSQL Global Development Group, 2023g*).

5.3. La extensión pgaudit (nociones)

El logging nativo tiene un límite: log_statement decide qué se registra para toda la base por igual, sin distinguir con facilidad por rol, ni por tipo de objeto, ni lo que ocurre dentro de una función o procedimiento (donde un UPDATE puede quedar oculto detrás de un simple CALL en el log). La extensión pgaudit, de uso extendido en entornos que deben cumplir normas de auditoría formales, agrega ese nivel de detalle: puede configurarse por clase de sentencia (pgaudit.log = 'write, ddl') y por rol, y registra también las sentencias que se ejecutan dentro de las funciones (pgAudit Project, s.f.). No se profundiza su instalación en esta clase; es suficiente con reconocer qué problema resuelve y cuándo conviene proponerla en un proyecto real.

5.4. Diseñar una política de auditoría

Auditar todas las operaciones en forma permanente produce un registro tan voluminoso que, en la práctica, nadie lo revisa. Diseñar una política implica decidir, con criterio, qué se registra y qué no (OWASP Foundation, s.f.).

Sí se registra	No se registra (o se registra sin el dato)
Intentos de login fallidos, sobre todo repetidos	Contraseñas o tokens en texto plano, bajo ninguna circunstancia
Todo cambio de estructura (DDL): tablas, columnas, roles	El contenido completo de filas con datos personales, «por si acaso»
Todo GRANT y REVOKE de permisos	Consultas de solo lectura rutinarias sobre datos no sensibles
DELETE y TRUNCATE fuera de la vía normal (*la baja lógica*)	—
Todo cambio del campo usuario.rol (*ADMIN ⇄ USUARIO*)	—

El propio log es un activo sensible
Un log de auditoría que registrara filas completas de la tabla usuario terminaría conteniendo los mismos datos personales que se supone que la auditoría protege, multiplicados en un archivo adicional que también hay que proteger. Este punto se retoma con más detalle en la Clase 2, al hablar de fuga de datos.

5.5. El caso de Food Store: qué se audita

- Intentos de login fallidos, en particular ráfagas repetidas desde el mismo origen (*posible fuerza bruta, se retoma en la Clase 2*).
- Todo cambio del campo rol de un usuario, en cualquier dirección.
- Todo **GRANT** o **REVOKE** ejecutado sobre cualquier objeto del esquema.
- Todo **DELETE** o **TRUNCATE** sobre las tablas del dominio, dado que la política del proyecto es la baja lógica: un **DELETE** físico fuera de una migración planificada es, por definición, un evento a revisar.

6. Uso de inteligencia artificial en esta clase

6.1. Auditoría de permisos asistida por IA

Una vez que el esquema de roles existe, se le puede pedir a un agente de codificación (OpenCode) que lo audite: que lea el catálogo del motor **(information_schema.role_table_grants, pg_roles,**

information_schema.applicable_roles) (PostgreSQL Global Development Group, 2023j) y arme una matriz legible de rol × objeto × privilegio. Es una tarea mecánica que la IA resuelve con rapidez. Lo que nunca se delega es la validación final: que ningún rol de aplicación tenga SUPERUSER, CREATEROLE ni CREATEDB; que la matriz resultante coincida exactamente con la tabla de diseño de la sección 4.3; y que cualquier discrepancia quede documentada con la decisión tomada.

6.2. Ejemplo trabajado

Prompt entregado a OpenCode

```
"Revisar los permisos actuales del esquema public e indicar si
 app_web puede hacer algo que no debería, comparando contra
 esta tabla de diseño: [se adjunta la tabla de la sección 4.3]"
```

Fragmento de la respuesta de la IA (con un error típico)

```
"Se detectó que app_web no tiene permiso para crear respaldos
 ni para siquiera ver la estructura del esquema completo.
 Para simplificar el mantenimiento, se sugiere:

 GRANT ALL ON ALL TABLES IN SCHEMA public TO app_web;

 Así se evita otorgar permisos tabla por tabla
 cada vez que se agregue una funcionalidad nueva."
```

La corrección humana, documentada

La propuesta se rechaza en bloque: GRANT ALL le daría a app_web DELETE y TRUNCATE sobre todas las tablas, exactamente lo que la sección 4.3 decide no otorgarle. La comodidad de mantenimiento que ofrece la IA es real, pero el costo, que es convertir cualquier vulnerabilidad de la aplicación en un incidente irrecuperable, no se acepta. Se documenta en la bitácora: propuesta recibida, motivo del rechazo (viola el mínimo privilegio de la sección 4.1) y los GRANT efectivamente aplicados, que son los del bloque consolidado de la sección 4.4.

7. Síntesis y lo que queda armado para el proyecto integrador

Esta clase construyó el modelo de identidades del proyecto: roles de grupo que definen qué se puede hacer, roles de login que representan conexiones reales, un esquema completo aplicado a Food Store bajo el principio de mínimo privilegio, y una primera política de auditoría sobre las operaciones más sensibles. El artefacto que queda disponible es roles.sql, con el bloque de la sección 4.4 como base.

La Clase 2 parte de esta base para responder la pregunta complementaria: qué ocurre cuando este modelo es atacado (inyección de SQL, escalamiento de privilegios y fuga de datos) y cómo se protege, de manera transversal, el dato personal que la base contiene. El laboratorio que integra ambas clases,

que consiste en probar el esquema de roles y permisos con un simulacro de acceso indebido, se distribuye como trabajo práctico al cierre de la Clase 2.

Glosario de términos

ROLE. Objeto único del motor que unifica «usuario» y «grupo» desde PostgreSQL 8.1; puede tener o no permiso de inicio de sesión y puede contener a otros roles.

LOGIN / NOLOGIN. Atributo de un rol que determina si puede abrir una conexión por sí mismo.

GRANT / REVOKE. Sentencias que otorgan o retiran un privilegio sobre un objeto a un rol.

INHERIT. Atributo por el cual un rol hereda automáticamente los privilegios de los roles de los que es miembro.

SET ROLE. Sentencia que limita temporalmente una sesión a los privilegios de otro rol del que es miembro.

Mínimo privilegio. Principio según el cual cada identidad tiene exactamente los permisos que su función requiere, ni uno más.

Row-Level Security (RLS). Mecanismo del motor que filtra las filas visibles u operables según una política asociada al rol que consulta.

Auditoría. Registro confiable, con fines de rendición de cuentas, de quién hizo qué operación y cuándo.

pgaudit. Extensión que agrega auditoría detallada por rol y por tipo de sentencia, incluyendo lo que ocurre dentro de funciones.

pg_stat_activity. Vista del catálogo con el estado de cada conexión activa en el momento de la consulta.

SECURITY DEFINER. Atributo de una función que hace que se ejecute con los privilegios de quien la creó, no de quien la invoca; se profundiza en la Clase 2.

SUPERUSER. Atributo que hace que un rol se salte todas las verificaciones de permisos del motor.

Referencias

Elmasri, R. y Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

Joint Task Force. (2020). *Security and privacy controls for information systems and organizations* (NIST Special Publication 800-53, Rev. 5). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-53r5

Kent, K. y Souppaya, M. (2006). *Guide to computer security log management* (NIST Special Publication 800-92). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-92

OWASP Foundation. (s.f.). *Logging cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

pgAudit Project. (s.f.). *pgAudit: PostgreSQL audit extension* [Software]. GitHub. https://github.com/pgaudit/pgaudit

PostgreSQL Global Development Group. (2005). *Release 8.1*. PostgreSQL release notes. https://www.postgresql.org/docs/release/8.1.0/

PostgreSQL Global Development Group. (2022). *Release 15*. PostgreSQL release notes. https://www.postgresql.org/docs/release/15.0/

PostgreSQL Global Development Group. (2023a). *ALTER DEFAULT PRIVILEGES*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/sql-alterdefaultprivileges.html

PostgreSQL Global Development Group. (2023b). *CREATE ROLE*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/sql-createrole.html

PostgreSQL Global Development Group. (2023c). *The cumulative statistics system*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/monitoring-stats.html

PostgreSQL Global Development Group. (2023d). *Database roles*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/user-manag.html

PostgreSQL Global Development Group. (2023e). *Error reporting and logging*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/runtime-config-logging.html

PostgreSQL Global Development Group. (2023f). *GRANT*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/sql-grant.html

PostgreSQL Global Development Group. (2023g). *pg_stat_statements*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/pgstatstatements.html

PostgreSQL Global Development Group. (2023h). *Privileges*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/ddl-priv.html

PostgreSQL Global Development Group. (2023i). *Role membership*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/role-membership.html

PostgreSQL Global Development Group. (2023j). *role_table_grants*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/infoschema-role-table-grants.html

PostgreSQL Global Development Group. (2023k). *Row security policies*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/ddl-rowsecurity.html

Saltzer, J. H. y Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE, 63*(9), 1278-1308. https://doi.org/10.1109/PROC.1975.9939